Indexation avec table de hachage (suite)

Exercice

Dans ce TP, vous continuez à implanter la solution d'indexation du TP précédent, si toutes les fonctionnalités n'ont pas encore été développées.

Si vous considérez que votre code supporte les fonctionnalités demandées en respectant les principes de la programmation fonctionnelle (pas de déclarations de `var`, préférer la récursion aux boucles `while`, `do-while` ou `for`, un minimum de fonctions non pures), vous allez alors enrichir votre programme avec les fonctionnalités suivantes.

Générateur de jeux de données aléatoires

Implantez un générateur de jeux de données aléatoires sur la base d'un ensemble de clés prédéfinies :

- vous disposez par exemple d'un ensemble d'une dizaine de clés ;
- vous tirez au hasard une valeur de clé avec une certaine fréquence, par exemple toutes les 2 à 3 secondes ;
- pour chaque clé sélectionnée, vous générez aléatoirement une valeur qui lui sera associée.

Ce couple clé/valeur doit être enregistré dans votre solution de stockage (fichier), et la clé doit être indexée dans la table de hachage.

Compaction basée sur la taille du fichier

L'approche de compaction ne doit plus suivre une solution basée sur une valeur de seuil (rapport pertinents/non-pertinents, cf. TP précédent), mais sur la **taille du fichier persisté** — dans votre cas, quelques Ko, suivant la taille des valeurs générées par votre générateur.

Lorsque cette limite est atteinte pour le fichier courant (nommé **active log** dans la suite du document) :

1. vous enregistrez celui-ci sur disque avec un nouveau nom composé du terme `log` et du timestamp d'enregistrement du fichier, par exemple `log-1728744295` ;
2. vous réinitialisez le fichier courant (l'active log) et vous mettez à jour l'index en conséquence.

La Figure 1 résume le comportement attendu de l'application.

Figure 1 : mise à jour de l'active log à la suite d'une nouvelle entrée

Avant — état de l'index et de l'active log :

clé	offset
abc	0
def	40
xyz	x
ghi	x+20

```
activeLog:
0     : abc -> blah ...
20    : def -> ...
40    : def -> ...
60    : xyz -> ...
80    : ghi -> ...
...   // des xyz et ghi
```

```
X     : xyz -> ...
x+20  : ghi -> "last log record"
```

À la suite de l'enregistrement de `ghi -> "last log record"`, un enregistrement `def -> "new log"` doit être enregistré, mais l'active log a atteint sa taille limite. Il faut alors :

1. archiver l'active log,
2. vider l'active log,
3. enregistrer la nouvelle clé/valeur dans celui-ci,
4. indexer la clé,
5. attendre de nouveaux enregistrements.

Après — nouvel état :

clé	offset
def	0

```
activeLog:
0 : def -> new log.


log-1728744295 (fichier archivé) :
0     : abc -> blah ...
20    : def -> ...
40    : def -> ...
60    : xyz -> ...
80    : ghi -> ...
...
x     : xyz -> ...
x+20  : ghi -> "last log record"
```

Requêtage

Vous devez supporter le requêtage depuis une valeur de clé :

- si la clé est dans l'index, on récupère efficacement sa valeur — c'est-à-dire en utilisant l'offset de l'index — dans l'active log ;
- si la clé n'est pas dans l'index, il faut rechercher la valeur la plus récente dans les logs archivés. Dans ce cas, il n'y a pas d'autre solution que de faire une recherche séquentielle dans les différents logs, en les considérant dans l'ordre chronologique **du plus récent au plus ancien**.

Expérimentation

Vous conduirez quelques expérimentations avec votre générateur, et mettrez en évidence (dans un fichier du zip de votre projet) la différence de performance — le temps de récupération de la valeur d'une clé — dans les situations où :

1. la valeur est dans l'active log,
2. la valeur n'est pas dans l'active log.

Rendu

Vous rendrez votre projet sur le site de e-learning en y incluant :

- le code source,
- le résultat des expérimentations